SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2 – CI/CD Pipeline Design Exercise

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO4	**Assessment:**	Assessment Tool 2 – CI/CD Pipeline Design Exercise
Weightage:	20%	**Total Marks:**	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:	E Ajay Velmail	**Reg. No.:**	192311385
Date:		**Duration:**	60 minutes

Code of Conduct

I Ajay Velmail E certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____E Ajay Velmail_____

Annexure A – CI/CD Pipeline Design Exercise

Instructions to Students:

Each student is assigned an individual problem statement. Based on the assigned problem statement, **design a CI/CD (Continuous Integration and Continuous Deployment) pipeline** for the proposed software system to automate the processes of code integration, building, testing, and deployment.

The exercise should include:

1. **Analyze the project requirements** and identify the CI/CD needs of the assigned problem statement.
2. **Design the CI/CD pipeline workflow** showing stages such as source code management, build, testing, code quality analysis, packaging, and deployment.
3. Identify suitable **tools and technologies** for each stage of the pipeline and justify their selection.
4. Define appropriate **automated testing strategies** to be integrated into the pipeline.
5. Design the **deployment strategy**, including development, testing/staging, and production environments.
6. Incorporate appropriate **security, quality checks, notifications, and rollback mechanisms** in the pipeline.

7. Prepare a **CI/CD pipeline diagram** and explain the workflow from code commit to deployment.

Deliverable: Submit a **CI/CD Pipeline Design document** containing the pipeline architecture/diagram, stages, tools, configuration/workflow, testing strategy, deployment process, and justification based on the assigned problem statement.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Requirement & Pipeline Analysis(15%)	Clearly analyzes the assigned problem and identifies comprehensive CI/CD requirements	Identifies most relevant requirements	Identifies basic requirements with some gaps	Requirements are unclear or incomplete
Pipeline Architecture & Workflow(25%)	Designs a complete, logical pipeline covering source, build, test, quality check, package, and deployment stages	Pipeline covers most major stages with minor gaps	Basic pipeline is designed but several stages are missing	Pipeline is incomplete or poorly structured
Tools & Technology Selection(20%)	Selects appropriate tools for each stage and provides clear justification	Appropriate tools selected with reasonable justification	Tools are identified but justification is limited	Tools are inappropriate or not clearly identified
Automated Testing & Quality Integration(15%)	Effectively integrates automated testing and code-quality/securi	Includes major testing and quality checks	Includes basic testing with limited integration	Testing/quality checks are missing or inappropriate

	ty checks into the pipeline			
Deployment, Security & Rollback Strategy(15%)	Clearly designs deployment environments, security practices, monitoring, and rollback strategy	Covers deployment and most security/rollback aspects	Basic deployment strategy with limited security/rollback details	Deployment strategy is incomplete or lacks security considerations
Pipeline Diagram & Documentation(10%)	Clear, accurate, professional diagram with excellent explanation and documentation	Well-presented diagram and documentation with minor issues	Adequate diagram/documentation but lacks clarity	Poorly presented, incomplete, or difficult to understand

Criteria	**Mark**
Requirement & Pipeline Analysis(15%)	/5
Pipeline Architecture & Workflow(25%)	/4
Tools & Technology Selection(20%)	/4
Automated Testing & Quality Integration(15%)	/4
Deployment, Security & Rollback Strategy(15%)	/4
Pipeline Diagram & Documentation(10%)	/4
TOTAL	/25

CI/CD Pipeline Design for Cold-Chain Logistics Integrity Platform

1. Introduction

The Cold-Chain Logistics Integrity Platform is a distributed IoT-based system designed to continuously monitor refrigerated shipments used in food and pharmaceutical distribution.

The platform collects real-time telemetry such as:

- Temperature
- Humidity
- GPS location
- Door status
- Refrigeration-unit health

The system must detect environmental excursions, predict possible shipment failures, notify stakeholders, and generate compliance reports.

Since the platform handles real-time IoT data and must support thousands of simultaneously connected vehicles, the CI/CD pipeline must provide reliable automated testing, security validation, scalable deployment, continuous monitoring, and rapid rollback.

The CI/CD design follows the assessment requirement to automate code integration, building, testing and deployment while incorporating quality, security, reliability and DevOps practices.

2. CI/CD Requirements

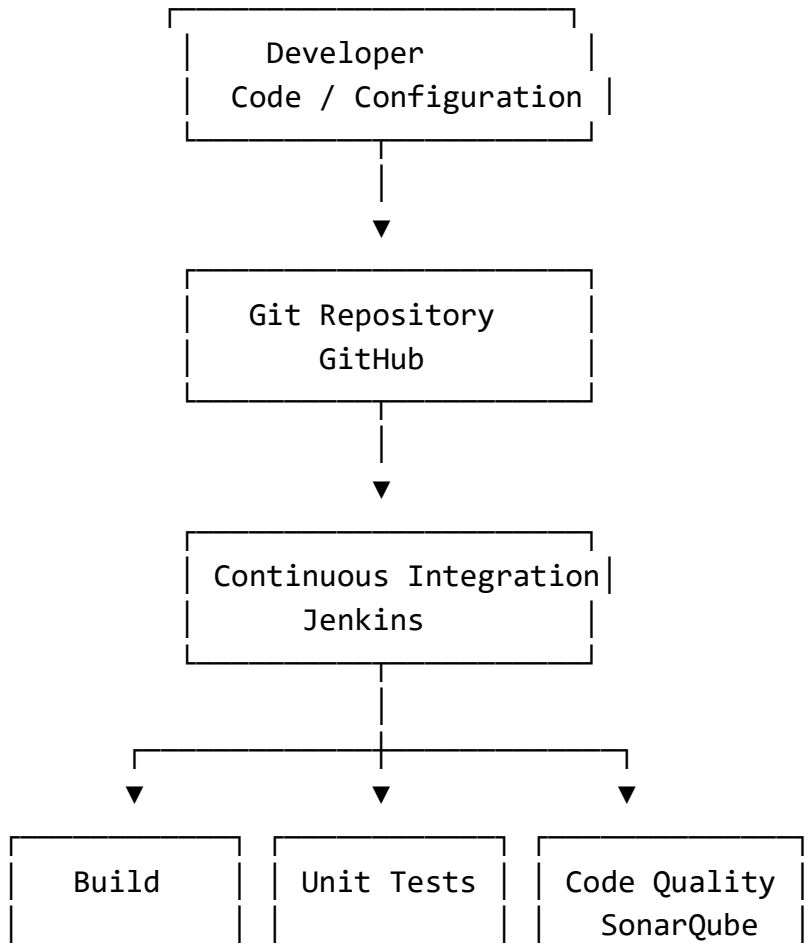
The major CI/CD requirements are:

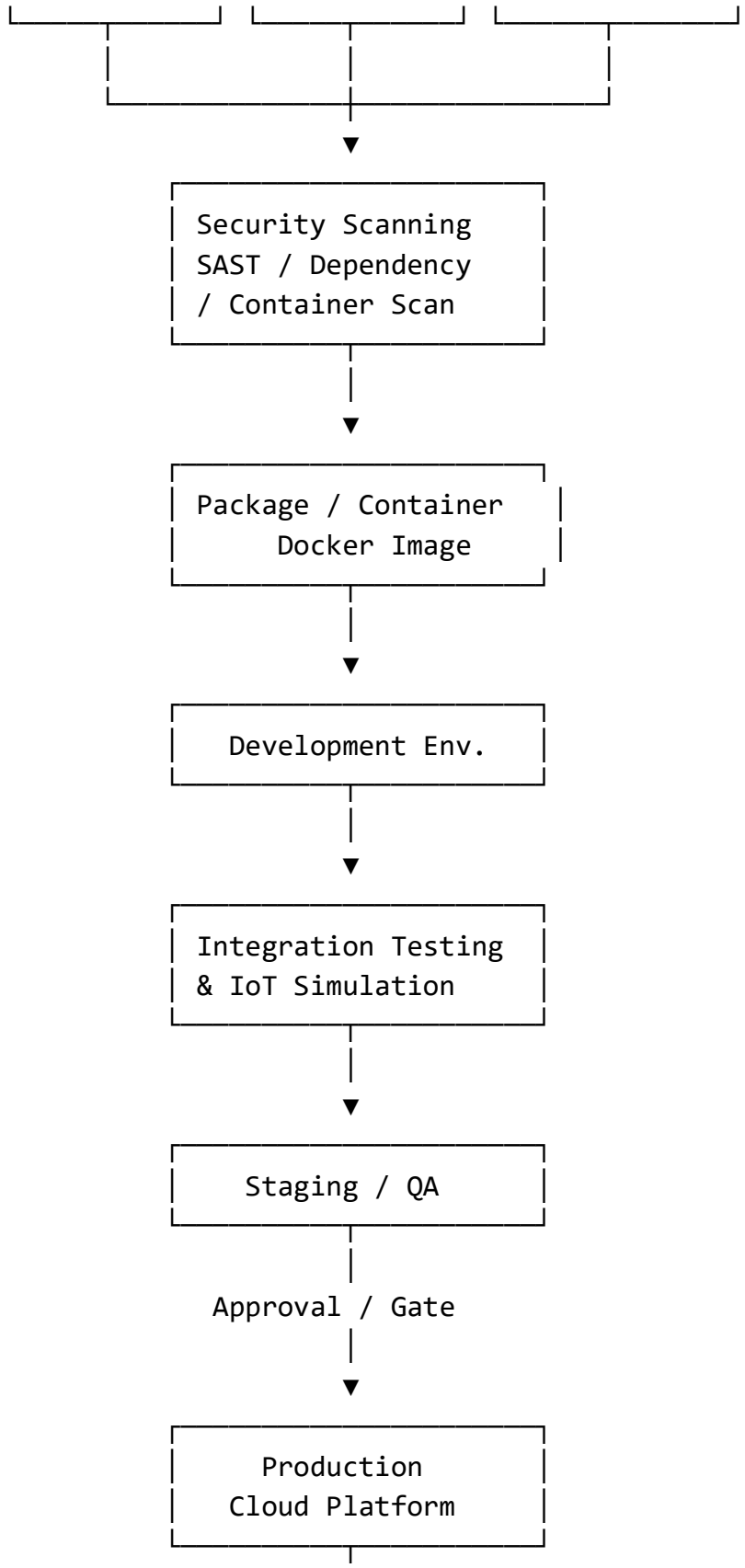
1. Automatic source-code integration.
2. Automated application builds.
3. Automated unit and integration testing.
4. IoT telemetry validation.
5. Automated code-quality analysis.
6. Security and dependency scanning.

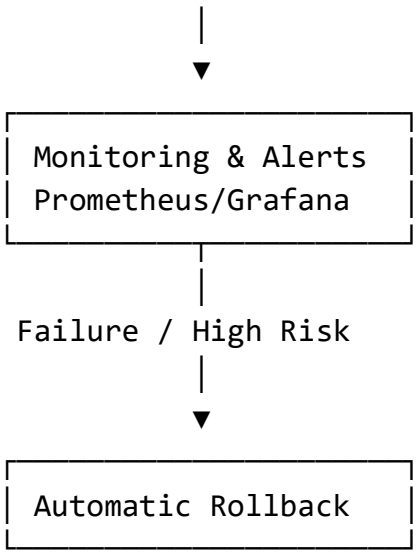
7. Container image creation.
8. Automated deployment to development.
9. Automated deployment/testing in staging.
10. Controlled deployment to production.
11. Monitoring after deployment.
12. Automatic rollback when a deployment fails.
13. Notifications to developers and operations teams.
14. Secure management of passwords, API keys and certificates.
15. Support for frequent releases without affecting running shipments.
16. High availability and reliability.

3. Proposed CI/CD Architecture

Pipeline Flow




Security Scanning
SAST / Dependency
/ Container Scan
Package / Container
Docker Image
Development Env.
Integration Testing
& IoT Simulation
Staging / QA
Approval / Gate
Production
Cloud Platform



This architecture covers the required source, build, testing, quality, packaging and deployment stages specified in the assessment.

4. Tools and Technologies

Pipeline Stage	Tool/Technology	Purpose
Source Code Management	GitHub	Store and manage source code
CI/CD Automation	Jenkins	Automate pipeline execution
Application Build	Maven/Gradle	Build backend application
Unit Testing	JUnit	Test individual application components
API Testing	Postman/Newman	Automate API testing
Integration Testing	JUnit/Testcontainers	Test interaction between services
Code Quality	SonarQube	Detect bugs, code smells and maintainability issues
Security Analysis	OWASP Dependency-Check	Detect vulnerable dependencies
Containerization	Docker	Package applications consistently
Container Security	Trivy	Scan container images
Container Registry	Docker Hub/Cloud Registry	Store approved container images
Deployment	Kubernetes	Deploy and scale services
Infrastructure	Cloud provider	Provide scalable infrastructure
Monitoring	Prometheus	Collect system metrics
Visualization	Grafana	Display system health and performance

Notifications	Email/Slack	Notify teams of pipeline results
Secrets	Cloud Secret Manager/Kubernetes Secrets	Protect credentials and keys

The assessment specifically requires suitable tools to be identified for each stage and their selection to be justified.

5. Pipeline Stages

Stage 1 – Source Code Management

Developers commit application changes to a GitHub repository.

Example branches:

```
main       → Production
develop    → Development
feature/* → Individual development
release/* → Production preparation
```

A pull request is required before merging changes into the main branches.

Checks

- Commit validation
- Pull-request review
- Branch protection
- Automated CI trigger

Stage 2 – Build

Jenkins automatically starts after a successful code commit or pull request.

The pipeline:

1. Checks out the source code.
2. Installs dependencies.
3. Compiles the application.
4. Builds required services.
5. Checks for build errors.

If the build fails, the pipeline stops and developers are notified.

6. Automated Testing Strategy

The Cold-Chain platform requires multiple levels of automated testing.

6.1 Unit Testing

Unit tests verify individual components.

Examples:

- Temperature-threshold calculation
- Humidity validation
- GPS validation
- Alert generation
- Shipment-status calculation
- Compliance-report calculations

Example:

```
Input Temperature = 5°C
Allowed Range = 2°C–8°C

Expected:
Temperature Status = NORMAL
Alert = FALSE
```

6.2 Boundary Testing

The pipeline should automatically test product-specific limits.

For a product requiring 2°C–8°C:

```
1.9°C → Excursion
2.0°C → Valid
2.1°C → Valid
7.9°C → Valid
8.0°C → Valid
8.1°C → Excursion
```

This is particularly important because incorrect threshold processing could result in unsafe food or pharmaceutical shipments.

6.3 Integration Testing

Integration tests verify communication between:

```
IoT Sensors
     ↓
IoT Gateway
     ↓
Telemetry Service
     ↓
Message Broker
     ↓
Processing Service
     ↓
Database
     ↓
Alert Service
```

Tests verify that telemetry is correctly received, processed, stored and forwarded to the appropriate services.

6.4 API Testing

API tests verify:

- User authentication
- Device registration
- Shipment creation
- Telemetry submission
- Threshold configuration
- Alert retrieval
- Report generation

Invalid requests must return appropriate error responses.

6.5 IoT Simulation Testing

Since thousands of vehicles may be connected, simulated devices can generate telemetry.

Example:

```
Vehicle V001 → Temperature = 5°C
Vehicle V002 → Temperature = 7°C
Vehicle V003 → Temperature = 9°C
Vehicle V004 → Temperature = 4°C
```

The system should correctly identify V003 as having a temperature excursion.

7. Code Quality Analysis

After automated tests, SonarQube performs static code analysis.

The pipeline checks for:

- Bugs
- Code smells
- Security issues
- Duplicate code
- Poor maintainability
- Test coverage

A quality gate should prevent deployment if critical quality requirements are not satisfied.

Example:

```
Quality Gate
     │
     ├── Passed → Continue
     │
     └── Failed → Stop Pipeline
```

8. Security Testing

Security checks are integrated directly into the CI/CD pipeline.

Security checks include:

1. Static Application Security Testing.
2. Dependency vulnerability scanning.
3. Docker image scanning.
4. Secret detection.
5. API authentication testing.
6. Authorization testing.
7. Secure communication verification.

Sensitive information such as:

```
Database Password
API Key
Cloud Credentials
Encryption Key
```

```
IoT Certificates
```

must never be stored directly in source code.

They should be stored using a secure secrets-management mechanism.

9. Containerization

Docker is used to package the platform's services.

Example services:

```
coldchain-api
telemetry-service
alert-service
prediction-service
report-service
dashboard
```

Each service can be packaged as a Docker image.

Example:

```
coldchain-api:v1.0.0
telemetry-service:v1.0.0
alert-service:v1.0.0
```

Images are scanned before being pushed to the container registry.

10. Development Environment

After successful build, testing and security checks, the application is automatically deployed to the development environment.

Purpose:

- Developer verification
- API testing
- IoT simulation
- Early integration testing

Test IoT devices can generate simulated temperature, humidity, GPS and refrigeration data.

11. Staging Environment

The staging environment should closely resemble production.

It is used for:

- End-to-end testing
- Performance testing
- Security testing
- Large-scale IoT simulation
- Compliance-report testing
- Failure-recovery testing

Example:

```
Development
     ↓
Integration Tests
     ↓
Staging
     ↓
Performance + Security Tests
     ↓
Approval
     ↓
Production
```

12. Production Deployment

After successful staging validation, the application is deployed to production.

Kubernetes can be used to manage application containers.

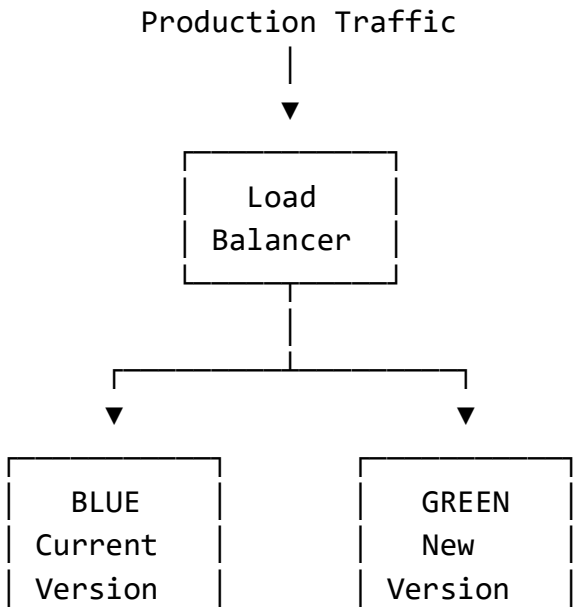
The production environment should use:

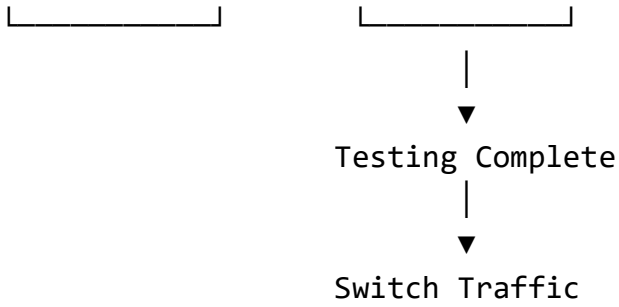
- Multiple application instances
- Load balancing
- Health checks
- Automatic scaling
- Replicated services
- Persistent storage
- Monitoring

This is important because the platform must support thousands of simultaneously connected vehicles.

13. Deployment Strategy

A **Blue-Green Deployment** strategy can be used.





The existing version remains available while the new version is tested.

If the new version works correctly, production traffic is switched to it.

If problems occur, traffic can immediately be returned to the previous version.

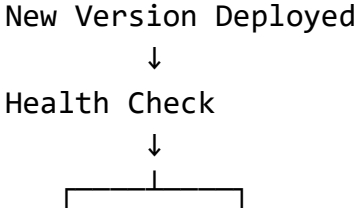
14. Rollback Strategy

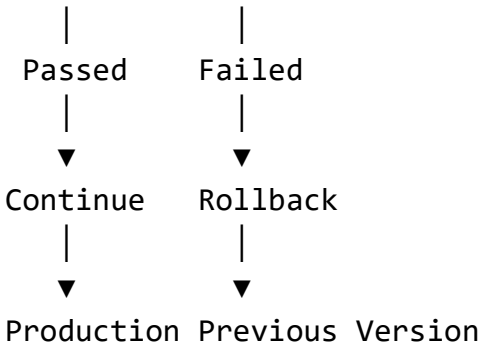
Rollback is essential because the platform is responsible for monitoring potentially critical shipments.

Rollback can be triggered when:

- Deployment fails.
- Health checks fail.
- Critical errors increase.
- API response time becomes unacceptable.
- Telemetry processing stops.
- Alert generation fails.
- Database connectivity fails.
- Monitoring detects abnormal application behavior.

Rollback Process





The previous stable container image should be retained so that it can be redeployed quickly.

15. Monitoring and Notifications

After production deployment, continuous monitoring is performed.

Application metrics

- CPU usage
- Memory usage
- API response time
- Error rate
- Request rate
- Telemetry processing rate

Cold-chain-specific metrics

- Number of active shipments
- Number of connected vehicles
- Temperature excursions
- Humidity excursions
- Refrigeration failures
- Sensor failures
- Delayed telemetry
- Alert-processing latency

Prometheus can collect metrics and Grafana can display dashboards.

Notifications can be sent to:

- Development team
- DevOps team
- Operations team
- Logistics administrators

Notifications should be generated for:

```
Build Failure
Test Failure
Security Failure
Deployment Failure
Production Failure
Rollback
Critical System Alert
```

16. CI/CD Security Controls

Security should be implemented throughout the pipeline.

Security Area	Control
Source code	Branch protection and code review
Authentication	Strong user authentication
Credentials	Secret management
Dependencies	Vulnerability scanning
Code	Static security analysis
Containers	Container image scanning
Network	TLS/secure communication
Deployment	Role-based access control
Production	Restricted deployment permissions
Audit	Deployment and access logs

17. Example Jenkins Pipeline Workflow

A simplified pipeline can follow:

```
1. Checkout Code
       ↓
2. Build
       ↓
3. Unit Tests
       ↓
4. Integration Tests
       ↓
5. SonarQube Quality Check
       ↓
6. Security Scan
       ↓
7. Build Docker Image
       ↓
8. Scan Docker Image
       ↓
9. Push Image to Registry
       ↓
10. Deploy to Development
       ↓
11. End-to-End Tests
       ↓
12. Deploy to Staging
       ↓
13. Performance/Security Tests
       ↓
14. Approval Gate
       ↓
15. Deploy to Production
       ↓
16. Health Check
       ↓
17. Monitoring
       ↓
```

18. Rollback if Required

18. Sample CI/CD Pipeline Configuration

A simplified Jenkins-style pipeline is:

```
pipeline
{
    agent any

    stages
    {
        stage('Checkout')
        {
            steps
            {
                checkout source code
            }
        }

        stage('Build')
        {
            steps
            {
                build application
            }
        }

        stage('Unit Test')
        {
            steps
            {
                run unit tests
            }
        }
```

```
stage('Integration Test')
{
    steps
    {
        run integration tests
    }
}

stage('Code Quality')
{
    steps
    {
        run SonarQube analysis
    }
}

stage('Security Scan')
{
    steps
    {
        scan dependencies
    }
}

stage('Package')
{
    steps
    {
        build Docker image
    }
}

stage('Container Scan')
{
    steps
    {
        scan Docker image
    }
}
```

```
    stage('Deploy Development')
    {
        steps
        {
            deploy to development
        }
    }

    stage('Deploy Staging')
    {
        steps
        {
            deploy to staging
        }
    }

    stage('Production')
    {
        steps
        {
            deploy to production
        }
    }
}

post
{
    success
    {
        notify successful deployment
    }

    failure
    {
        notify failure
        rollback if required
    }
}
```

```
}
```

19. Quality Gates

The pipeline should not allow every commit to reach production.

Gate 1 – Build

```
Build successful?
YES → Continue
NO  → Stop
```

Gate 2 – Testing

```
Automated tests passed?
YES → Continue
NO  → Stop
```

Gate 3 – Quality

```
SonarQube quality gate passed?
YES → Continue
NO  → Stop
```

Gate 4 – Security

```
Critical vulnerabilities?
NO  → Continue
YES → Stop
```

Gate 5 – Staging

```
Staging tests passed?
YES → Production
NO  → Stop
```

20. Handling IoT-Specific Failures

The CI/CD pipeline should also test failure conditions relevant to the platform.

Sensor Failure

```
Sensor stops transmitting
        ↓
Telemetry timeout detected
        ↓
Sensor marked unhealthy
        ↓
Alert generated
```

Network Failure

```
Vehicle loses network
        ↓
Telemetry temporarily unavailable
        ↓
System detects missing data
        ↓
Connection restored
        ↓
Buffered data processed
```

Refrigeration Failure

```
Refrigeration unit failure
        ↓
Temperature begins increasing
        ↓
Monitoring service detects excursion
        ↓
Alert generated
        ↓
Stakeholder notified
```

These scenarios should be included in automated integration and end-to-end testing.

21. Benefits of the Proposed CI/CD Pipeline

Faster Development

Automated builds and tests reduce manual effort.

Improved Quality

Every code change passes automated testing and quality checks.

Better Security

Security scanning identifies vulnerabilities before production deployment.

Higher Reliability

Automated monitoring and rollback reduce the impact of failed releases.

Scalability

Containerization and Kubernetes allow services to scale according to demand.

Faster Recovery

Blue-green deployment and rollback allow the previous stable version to be restored quickly.

Continuous Delivery

Validated changes can be delivered rapidly while maintaining quality and reliability.

22. Requirement-to-Pipeline Mapping

Platform Requirement	CI/CD Solution
Real-time telemetry	Integration/API testing
Temperature monitoring	Unit + boundary testing
Humidity monitoring	Unit + integration testing
GPS tracking	API/integration testing
Door status	Automated functional testing
Refrigeration monitoring	Failure testing
Product-specific thresholds	Boundary testing
Automatic alerts	Integration/end-to-end testing
Failure prediction	Model/service testing
Compliance reports	Automated report testing
Thousands of vehicles	Load/scalability testing
Fault tolerance	Failure/recovery testing
Secure transmission	Security testing
High availability	Kubernetes/health checks
Continuous deployment	Jenkins pipeline
Rollback	Blue-green deployment

23. Conclusion

The proposed CI/CD pipeline provides an automated and secure workflow for developing, testing and deploying the Cold-Chain Logistics Integrity Platform.

The pipeline integrates **GitHub, Jenkins, automated testing, SonarQube, security scanning, Docker, Kubernetes, monitoring and rollback mechanisms**. It validates both normal

application behavior and critical cold-chain scenarios such as temperature excursions, sensor failures, refrigeration failures and network interruptions.

Development, staging and production environments are separated to reduce deployment risk. Automated quality and security gates prevent defective or vulnerable builds from reaching production. Blue-green deployment and rollback mechanisms provide additional reliability.

Therefore, the proposed CI/CD architecture supports the platform's key requirements of **scalability, security, reliability, fault tolerance and high availability**, while enabling rapid and controlled software delivery.

Expected Result

The designed CI/CD pipeline successfully automates the complete software delivery lifecycle:

Code Commit → Build → Test → Quality Analysis → Security Scan → Package → Development → Staging → Approval → Production → Monitoring → Rollback if Required

This satisfies the assessment's required CI/CD architecture, automated testing, tool selection, deployment, security, rollback and documentation components.